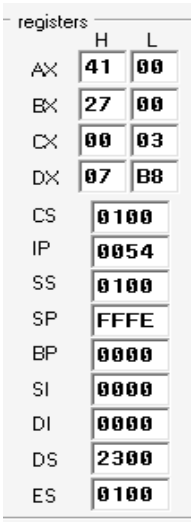
FA23-BCE-113
Muhammad Ahmad
MSI-Lab # 2

Task 1: Fill the table after compiling the code
Type in the following program in editor of EMU8086 and fill in the table.

Instruction	AH	AL	BH	BL	DH	DL	CL	CF	DS
MOV AX,1456	05	B0						0	
MOV BX,0FF00H			FF	00				0	
XOR AX,AX	00	00						0	
MOV BX,2300H			23	00					
MOV DS,BX									2300
MOV [1601H],25H								2300:1601 -> 25H	
OR BH,[1601H] 23,25			27						
MOV CL,3							3		
MOV DX,23DCH					23	DC			
SHL DX,CL					1E	E0		1	
SHR DX,3					03	DC			
SAL DX,1					07	B8			
MOV AH,250	FA							0	
ADD AH,10	04							1	
RCR AH,2	41	00	27	00	07	B8	3	0	2300



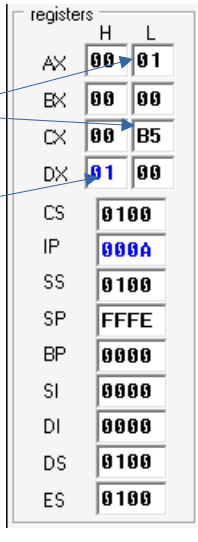
Description: The table is filled in an incremental way, assuming all register content in hex, like at every step, value of registers are updated according to the instruction given, the new value is written to register when it is overwritten by the new register, at the end, all upgraded values are written in one row, also the screenshot of final register content is also attached above

Task 2: Write and assembly program to count the number of 1's
Write an assembly language program that counts the number of '1's in a byte residing in CL register. Store the counted number in DH register.

```
MOV CL, 10110101B   ; Example byte
XOR DH, DH          ; DH = 0 (counter)
; Check bit0
MOV AL, CL
AND AL, 01H
ADD DH, AL
```

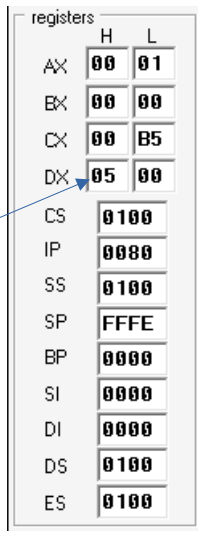
LSB is 1

Add 1 to DH



```
; Check bit1
MOV AL, CL
SHR AL, 1
AND AL, 01H
ADD DH, AL
; Check bit2
MOV AL, CL
SHR AL, 2
AND AL, 01H
ADD DH, AL
; Check bit3
MOV AL, CL
SHR AL, 3
AND AL, 01H
ADD DH, AL
; Check bit4
MOV AL, CL
SHR AL, 4
AND AL, 01H
ADD DH, AL
; Check bit5
MOV AL, CL
SHR AL, 5
AND AL, 01H
ADD DH, AL
; Check bit6
MOV AL, CL
SHR AL, 6
AND AL, 01H
ADD DH, AL
; Check bit7
MOV AL, CL
SHR AL, 7
AND AL, 01H
ADD DH, AL
```

Final DX value is 5



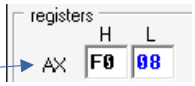
Description: We have to counts the number of '1's in a byte residing in CL register, we mov CL in a helping register AL ,and then and the LSB with 1 which results AL to 0 if LSB is 0 and vice versa. Add the result in DH {`ADD DH, AL`}. At every step we shift right {`SHR AL, 1`} so that the next bit is now checked , we repeat it 8 times (0-7) so that we check all bits.

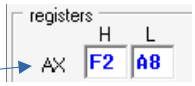
Task 3: Bits' manipulation

Write an assembly language program to perform the following tasks:
a) Set the leftmost 4 bits of AX
b) Clear the rightmost 3 bits of AX
c) Invert the bits 5,7 and 9 of AX.

```
MOV AX, 000FH

; a) Set the leftmost 4 bits of AX
;  1111 0000 0000 0000 ->  F000H
OR AX, 0F000H
; b) Clear the rightmost 3 bits of AX
;  1111 1111 1111 1000 ->  FFF8H
AND AX, 0FFF8H
; c) Invert the bits 5,7 AND 9 of AX.
; assuming bits (15 – 0)
; 0000 0010 1010 0000 -> 02A0H
XOR AX, 02A0H
```





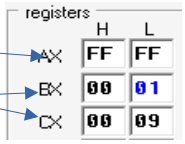
Description:

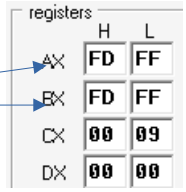
To set(1) the bit we or the number with 1 and or rest of the bit with 0.
To clear(0) the bit we and it with 0 , and rest of the bits with 1.
To invert a bit we xor the bit with one and xor rest of the bit with 0.

Task 4: Bits' masking

Write an assembly language program that clears any bit (from bit0 to bit15) in AX register, leaving
other bits unchanged. Number of bit that is to be cleared is stored in CL register.
Hint: If the number 9 is stored in CL register, it means 9th bit of AX should be cleared

```
MOV AX, 0FFFFH
MOV CL, 9   ; 9th bit to clear
MOV BX, 1
SHL BX, CL  ; Mask = 1 << CL
NOT BX      ; Invert mask
AND AX, BX  ; Clear that bit
```





Description: To clear(0) a bit at specific place we have to and it with 0, rest of the bits and with 1. So we take a helping register BX and set first bit and shift left with count of the bit which we have to clear, at last we invert the helping register so that the mth bit will clear and rest are unchanged. AND the given value with helping register BX for masking